Name of the Student	**K Ashok**
Reg. No	**192425322**
GitHub Link	

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**
SLOT : D

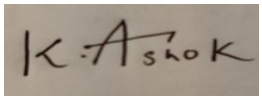
COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25% **Duration: 90 minutes**
Assessment Tool Weightage: 25% **Marks: 25**

Code of Conduct:

I K Ashok / 192425322 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student:K Ashok**

Criteria	**Conceptual Understanding (2.5)**	Linkages **(2.5)**	Organization **(2)**	Integration of Literature **(2)**	**Clarity & Presentation *(1)***	**Total (10)**
Weightage	***25 %***	***25%***	***20%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS: 02 **Total Marks: 20**

Case Study Question 1: CNN for Automated Medical Image Classification (10 Marks)
A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

1. Suitable CNN Architecture for Medical Image Classification

A suitable CNN architecture can be designed as follows:

Input Layer
- Input: Chest X-ray image, for example 224 x 224 x 1 for a grayscale image.
- Images can be resized to a fixed size and normalized before being given to the CNN.

Convolutional Block 1
- Convolution layer: 32 filters of size 3 x 3.
- ReLU activation function.
- Batch Normalization.
- Max Pooling: 2 x 2.

Convolutional Block 2
- Convolution layer: 64 filters of size 3 x 3.
- ReLU activation.
- Batch Normalization.
- Max Pooling: 2 x 2.

Convolutional Block 3
- Convolution layer: 128 filters of size 3 x 3.
- ReLU activation.
- Batch Normalization.
- Max Pooling: 2 x 2.

Convolutional Block 4
- Convolution layer: 256 filters of size 3 x 3.
- ReLU activation.
- Batch Normalization.
- Max Pooling: 2 x 2.

Classification Section
- Flatten or Global Average Pooling.
- Fully connected layer with around 128 neurons.
- Dropout layer, for example 0.5.
- Final output layer with 2 neurons for normal and disease-affected classes.
- Softmax activation for obtaining class probabilities.

The network progressively learns simple features in the early layers and more complex medical features in deeper layers. The final layers use these learned features to classify the X-ray.

2. Motivation for CNNs and Role of Different Layers

CNNs are suitable for medical image classification because they are specifically designed to process image data. They can automatically learn useful features such as edges, textures, shapes, patterns and abnormal regions without requiring manual feature engineering.

Convolutional Layers:
Convolutional layers apply filters to small regions of the image. The filters detect useful visual patterns. Early layers may detect edges and simple textures, while deeper layers detect more complex patterns such as anatomical structures and disease-related abnormalities.

3. Parameter Sharing in CNNs

In a fully connected network, every neuron is connected to every input pixel. This can create a very large number of trainable parameters.

For example, an image with 224 x 224 pixels has 50,176 pixels. Connecting all of these pixels directly to 1,000 neurons would require approximately:

50,176 x 1,000 = 50,176,000 weights

This is a very large number.

CNNs use parameter sharing. A single filter is applied to many different locations of the image. The same filter weights are reused across the entire image.

For example, a 3 x 3 filter has only 9 weights plus a bias, regardless of the image size. If 32 such filters are used, the number of weights in the convolution layer is approximately:

3 x 3 x 1 x 32 = 288 weights, plus biases.

Therefore, parameter sharing greatly reduces the number of trainable parameters, memory requirements and computational cost. It also allows the CNN to detect the same feature at different locations in an image.

4. Regularization Techniques to Reduce Overfitting

Medical image datasets may contain thousands of images, but they may still be insufficient compared with the large number of parameters in a deep neural network. Therefore, overfitting is an important concern.

Data Augmentation:
Training images can be randomly transformed using:
- Rotation
- Horizontal flipping when medically appropriate
- Cropping
- Scaling
- Translation
- Small brightness and contrast changes
- Controlled noise

Data augmentation creates more variations of training images and helps the model generalize to X-rays from different machines and conditions.

Dropout:
Dropout randomly deactivates some neurons during training. For example, a dropout rate of 0.5 can be used before the final classification layer. This prevents the network from depending too strongly on particular neurons.

Batch Normalization:
Batch normalization normalizes intermediate activations during training. It helps stabilize and speed up training and can also provide a regularization effect.

Other useful techniques include:
- Early stopping
- L2 regularization/weight decay
- Transfer learning
- Cross-validation

5. AlexNet vs ResNet for Medical Image Classification

AlexNet:
- Relatively shallow compared with modern CNNs.
- Around 8 learnable layers.
- Historically important and demonstrated the effectiveness of deep CNNs.
- Easier to understand and computationally simpler.
- However, it has fewer layers and generally weaker feature representation than modern architectures.

ResNet:
- Uses residual/skip connections.

- Can be much deeper, such as ResNet-18, ResNet-34, ResNet-50 and deeper versions.
- Skip connections help gradients flow through the network.
- Supports learning of complex and hierarchical image features.
- Usually provides stronger image classification performance than AlexNet.
- Pretrained ResNet models can be fine-tuned on medical X-ray datasets.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition (10 Marks)
An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.
Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

1. CNN Architecture for Autonomous Vehicle Object Recognition

A CNN-based system can be designed with the following stages:

Input Layer:
- Camera images are captured from the vehicle.
- Images are resized to a fixed resolution such as 224 x 224 or another resolution suitable for the selected model.
- Pixel values are normalized.

Convolutional Feature Extraction:
- Several convolutional layers extract visual features.
- Early layers detect edges, corners and simple textures.
- Middle layers detect shapes and parts of objects.
- Deeper layers learn high-level representations of cars, pedestrians, signs, bicycles and obstacles.

Activation:
- ReLU activation is used after convolutional layers to introduce non-linearity.

Pooling:
- Max pooling or other downsampling methods reduce spatial dimensions and computational cost.

Deep Feature Layers:
- Multiple convolutional blocks learn increasingly complex features.

Output:
For simple image classification, fully connected/output layers can classify the image into categories such as car, pedestrian, bicycle, traffic sign or obstacle.

For a real autonomous vehicle, object detection is generally more suitable than simple image classification because the system must identify both the object class and its location in the image. CNN-based detection architectures can therefore be used together with suitable detection heads.

2. Purpose of the Main Layers and Progressive Feature Learning

Input Layer:
The input layer receives images captured by vehicle cameras.

Convolutional Layer:
Convolutional layers use filters to scan the image and detect local patterns.

Activation Layer:
ReLU introduces non-linearity, allowing the network to learn complex relationships instead of only simple linear patterns.

Pooling Layer:
Pooling reduces feature-map size and computation. It also helps the model become less sensitive to small changes in object position.

Fully Connected/Output Layers:
These layers use the high-level features extracted by the CNN to make classification decisions.

Progressive Feature Learning:

Layer 1:
The network learns low-level features such as:
- Edges
- Lines
- Corners
- Basic color/brightness patterns

Middle layers:
The network learns:
- Shapes
- Curves

- Object parts
- Textures

Deep layers:
The network learns high-level concepts such as:
- Car-like structures
- Human/pedestrian shapes
- Traffic signs
- Bicycles
- Road obstacles

Thus, CNNs automatically transform raw pixels into increasingly meaningful representations.

3. Importance of Parameter Sharing

Parameter sharing means that the same convolutional filter is used at different locations of an image.

For example, a filter that learns to detect a vertical edge can detect that edge on the left, center or right side of an image using the same weights.

This provides several benefits:

Reduced Number of Parameters:
A CNN does not need separate weights for every image location.

Reduced Computational and Memory Requirements:
Fewer parameters mean less memory usage and generally more efficient computation.

Translation-Related Feature Detection:
An object or feature can appear at different positions in the image. Because the same filter scans the whole image, the CNN can recognize similar features even when their locations change.

Better Generalization:
The network learns reusable visual patterns instead of memorizing a specific position of an object.

This is particularly important in autonomous vehicles because cars, pedestrians and signs can appear at different positions in camera images.

4. Overfitting and Suitable Regularization Methods

Overfitting occurs when the CNN performs very well on training images but poorly on unseen road images.

Autonomous vehicle images can vary significantly because of:
- Day and night conditions
- Rain
- Fog
- Shadows

- Different roads
- Different camera angles
- Different object distances
- Different weather conditions

Suitable regularization techniques include:

Data Augmentation:
Training images can be modified using:
- Rotation
- Cropping
- Scaling
- Translation
- Brightness changes
- Contrast changes
- Noise
- Weather simulation
- Appropriate horizontal flipping

This helps the model learn robust features.

Dropout:
Dropout can be used in suitable parts of the network, particularly in classification layers, to reduce dependence on individual neurons.

Batch Normalization:
Batch normalization stabilizes activations and makes training more stable. It may also reduce overfitting.

Weight Decay/L2 Regularization:
This penalizes very large weights and encourages simpler models.

Early Stopping:
Training can be stopped when validation performance stops improving.

Transfer Learning:
A pretrained CNN can be fine-tuned on vehicle-specific datasets. This can improve performance when the available labeled dataset is limited.

5. AlexNet vs ResNet

AlexNet:
- Introduced in 2012 and has about 8 learnable layers.
- Uses convolutional layers followed by fully connected layers.
- Uses ReLU activation.
- Uses pooling to reduce feature-map dimensions.
- Relatively simple compared with modern deep CNNs.
- Easier to train than very deep networks.
- Has a large fully connected component and is less parameter-efficient than many newer architectures.

- Provides good basic feature learning but has limited depth compared with ResNet.

ResNet:
- Uses residual blocks and skip connections.
- Available in different depths such as ResNet-18, ResNet-34, ResNet-50 and deeper versions.
- Skip connections allow information and gradients to flow more easily through the network.
- Reduces the vanishing-gradient problem that can occur in very deep networks.
- Can learn more complex and hierarchical features.
- Usually provides stronger image recognition performance than AlexNet.
- ResNet-18 or ResNet-34 can provide a useful balance between accuracy and computational cost.

Training Difficulty:
AlexNet is simpler, while very deep conventional networks can become difficult to train. ResNet solves much of this problem through residual connections.

Feature Learning:
ResNet generally learns richer and more discriminative features because it can use deeper architectures effectively.

Parameter Efficiency:
ResNet architectures can achieve strong performance without relying on the large fully connected layers used by older architectures such as AlexNet.

6. Recommended Architecture for Autonomous Vehicles

ResNet is generally the better choice than AlexNet for an autonomous vehicle vision system.

Reasons:

Accuracy:
Autonomous vehicles need highly reliable recognition of pedestrians, vehicles, signs and obstacles. ResNet's deeper feature learning can provide stronger recognition performance.

Feature Learning:
ResNet can learn both low-level and complex high-level visual patterns through many convolutional layers.

Vanishing Gradient:
Residual/skip connections help gradients flow through deep networks, making deeper feature extraction practical.

Computational Cost:
Very deep ResNet models can be computationally expensive. Therefore, the choice should depend on the vehicle's hardware. ResNet-18 or ResNet-34 can be considered when real-time processing is important, while larger versions can be used when more computational power is available.

Real-Time Requirement:

An autonomous vehicle needs low latency. A smaller ResNet model or a hardware-optimized version is often a better compromise than using a very deep network.

Important Practical Point:
For a real autonomous vehicle, simple image classification is not enough. The system normally needs object detection, because it must determine:
1. What the object is.
2. Where the object is located.
3. Potentially how large/close the object is.

Therefore, a ResNet-based feature extractor can be combined with an appropriate object detection architecture.

Final Recommendation:
A lightweight ResNet such as ResNet-18 or ResNet-34 is a strong choice when balancing accuracy, feature-learning capability and real-time computational requirements. If the hardware can support it and higher accuracy is required, deeper ResNet variants can be considered.

CONCLUSION

CNNs are highly suitable for both medical image analysis and autonomous vehicle vision because they automatically learn spatial and visual features directly from images.

Convolutional filters detect local patterns, pooling reduces spatial dimensions, activation functions introduce non-linearity, and deeper layers learn increasingly complex features. Parameter sharing significantly reduces the number of trainable parameters and allows the same feature detector to work at different image locations.

AlexNet is historically important and relatively simple, but ResNet is generally more suitable for modern applications because it supports deeper feature learning and uses residual connections to reduce training difficulties caused by vanishing gradients.

For medical X-ray classification, ResNet with transfer learning and strong regularization is a suitable choice. For autonomous vehicles, a lightweight ResNet-based model is preferable when real-time performance is required, although an object detection architecture is needed for complete vehicle perception.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand